

CSCI 375

Assignment: Starter 3 Code Review

Jasmine Monari

Overview

This assignment was a peer code review for a Boggle Solver written in Python. The solver takes a 2D letter grid and a dictionary as input and returns all valid words of 3 or more characters found by traversing adjacent tiles without reusing the same cell. My approach was to verify the core logic first, then address performance, style, and edge cases identified during review.

Feedback You Received

My partner said the code was functional and readable but flagged the prefix checking inside `find_all_words` as the biggest concern — every recursive call iterated through the entire dictionary using `startswith(word)`, which is $O(n)$ per call. The suggested fix was a trie. They also pointed out that `find_all_words` should be renamed to `_find_all_words`, that the method signature was passing `grid`, `fast_dictionary`, and `solution` as arguments unnecessarily since they're accessible through `self`, and that `self.solutions` should be initialized in `__init__` rather than inside `getSolution()`. The Qu tile edge case was also mentioned since it represents one tile but two characters.

Feedback You Gave (Partner: Christopher Reid)

My partner's code was well-structured and easy to read. I liked the use of setters for validation, separation of functionality into individual methods, and the clean 8-direction DFS traversal with proper backtracking. My main suggestions were to raise `ValueError` exceptions instead of failing silently or setting values like `cols` to `None`, since that could quietly pass edge cases without stopping the program. I also flagged that `self.dictionary` was redundant in `getSolution()` since word lookups already referenced `self.words`, and suggested adding a check for numeric or symbol elements in the grid.

Improvements You Implemented

The biggest change was replacing the $O(n)$ prefix loop with a trie. I wrote `_build_trie()` to construct a nested dictionary once at the start of `getSolution()`, where '\$' marks valid word endings. The recursive method now traverses the trie node by node and exits immediately when a character isn't found. I also renamed `find_all_words` to `_find_all_words`, removed the unnecessary parameters from its signature, moved `self.solutions` and `self.trie` into `__init__`, and added explicit Qu tile handling so both characters are contributed to the word and two trie levels are traversed.

Static Analysis Results

In `boggle_solver.py` the main issues were 2-space indentation instead of 4, inline comment formatting, lines over 79 characters, and an unused import `re`. In `tests.py` the issues were more widespread — mixed tabs and spaces throughout, semicolons at the end of lines, incorrect import ordering triggering E402, and missing blank lines between class definitions. All issues were resolved by standardizing 4-space indentation, removing semicolons, reordering imports, and fixing comment formatting.

Regression Testing

After refactoring I re-ran the full test suite to confirm the trie implementation didn't change which words were found. The suite covers scalability across grid sizes, edge cases like empty and 1x1 grids, cell reuse prevention, complex traversal paths, and Qu/St tile handling. 25 of 26 tests passed after all fixes were applied.

Reflection

This process showed me that working code and well-structured code aren't the same thing. The prefix performance issue was easy to miss with small test inputs but would compound quickly at scale. Implementing the trie pushed me to think about data structure choices earlier rather than optimizing after the fact. Managing the Git workflow — resolving a non-fast-forward push error and keeping commits clean — was also a reminder that version control discipline matters as much as the code itself in a team setting.

